

TEAM 9: REFACTORING

COURSE: SOEN 6441 (WINTER 21)

INSTRUCTOR- JOEY PAQUET

TEAM MEMBERS:

1. Jayasurya Pazhani
2. Jayanth Apagundi
3. Shuvanidhi Suresh
4. Shahul Hameed P T S
5. Vinisha Kora Venkatesalu

Potential Refactoring Targets:

The following list of refactoring targets have been taken mainly from the new requirements established in build 2, and based on pain points and inconsistencies encountered during the development of build 1.

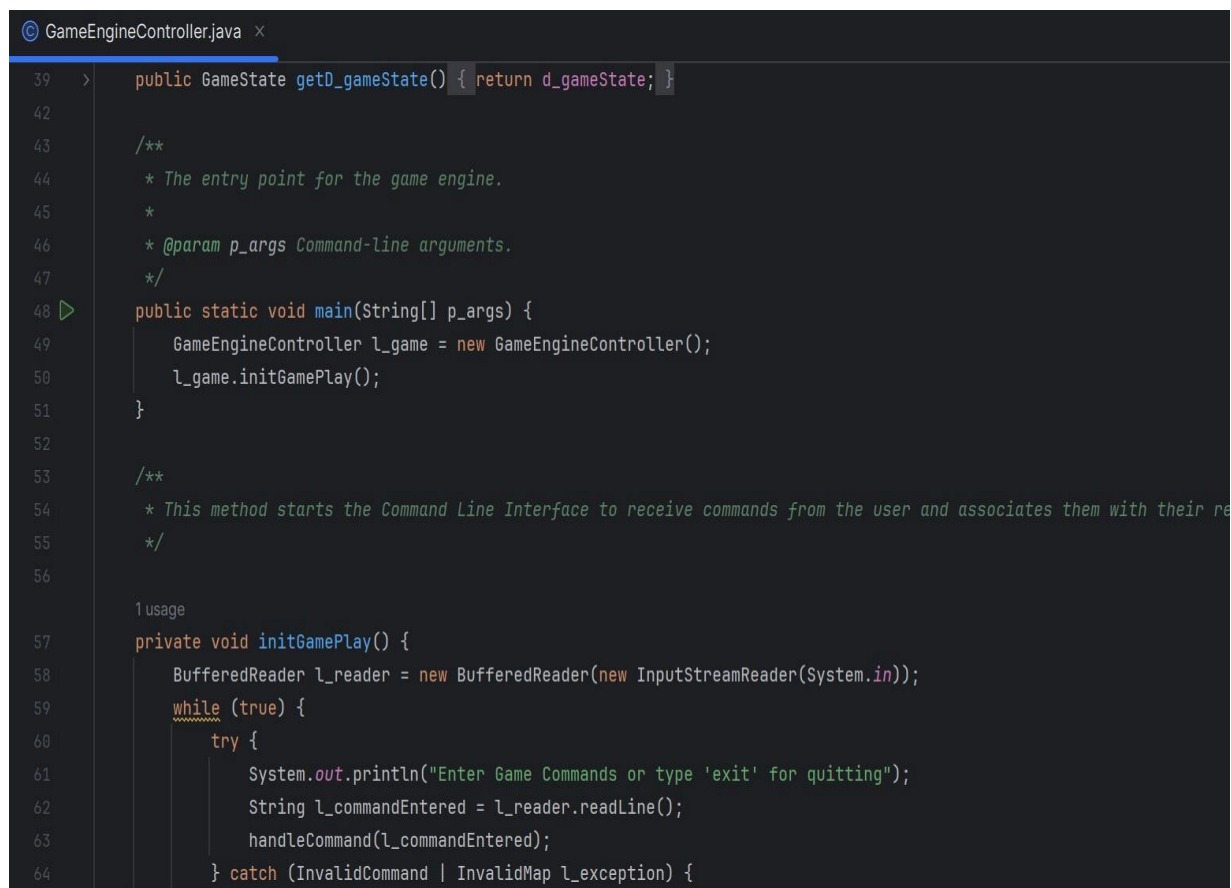
1. Implement the phases of the application, including the phases using State pattern.
2. Implementation of a game log file using the Observer pattern.
3. Use Command pattern to implement the Orders.
4. Identify and decouple dependencies.
5. Enhancing Exception Handling.
6. Improve Code Coverage with Additional Tests.
7. Encapsulate data members with appropriate access modifiers to enforce data integrity and information hiding.
8. Decompose large methods or classes into smaller, cohesive components to improve code organization.
9. Ensure consistent formatting and use meaningful names to enhance code readability Change Continent Check in Map validation.
10. Optimizing loops for better performance.
11. Improve User Interaction.
12. Organize code into logical sections, and group related methods together
13. Remove Redundant code blocks.
14. Check for unused variables, methods or classes.
15. Simplify complex conditional logic.

Actual Refactoring Targets:

The list of actual refactoring taken from the target list above were chosen mainly because of the new requirements established in build 2, and on the greatest pain points and inconsistencies encountered during the development of build 1.

1. Implement the phases of the application, including the phases using State pattern

Before: The code had a monolithic structure where different phases of the game (e.g., startup, issue order, order execution) were handled within the same methods or classes.



```
GameEngineController.java x
39 > public GameState getD_gameState() { return d_gameState; }
42
43 /**
44  * The entry point for the game engine.
45  *
46  * @param p_args Command-line arguments.
47  */
48 public static void main(String[] p_args) {
49     GameEngineController l_game = new GameEngineController();
50     l_game.initGamePlay();
51 }
52
53 /**
54  * This method starts the Command Line Interface to receive commands from the user and associates them with their re
55  */
56
57 1 usage
58 private void initGamePlay() {
59     BufferedReader l_reader = new BufferedReader(new InputStreamReader(System.in));
60     while (true) {
61         try {
62             System.out.println("Enter Game Commands or type 'exit' for quitting");
63             String l_commandEntered = l_reader.readLine();
64             handleCommand(l_commandEntered);
65         } catch (InvalidCommand | InvalidMap l_exception) {
```

Before

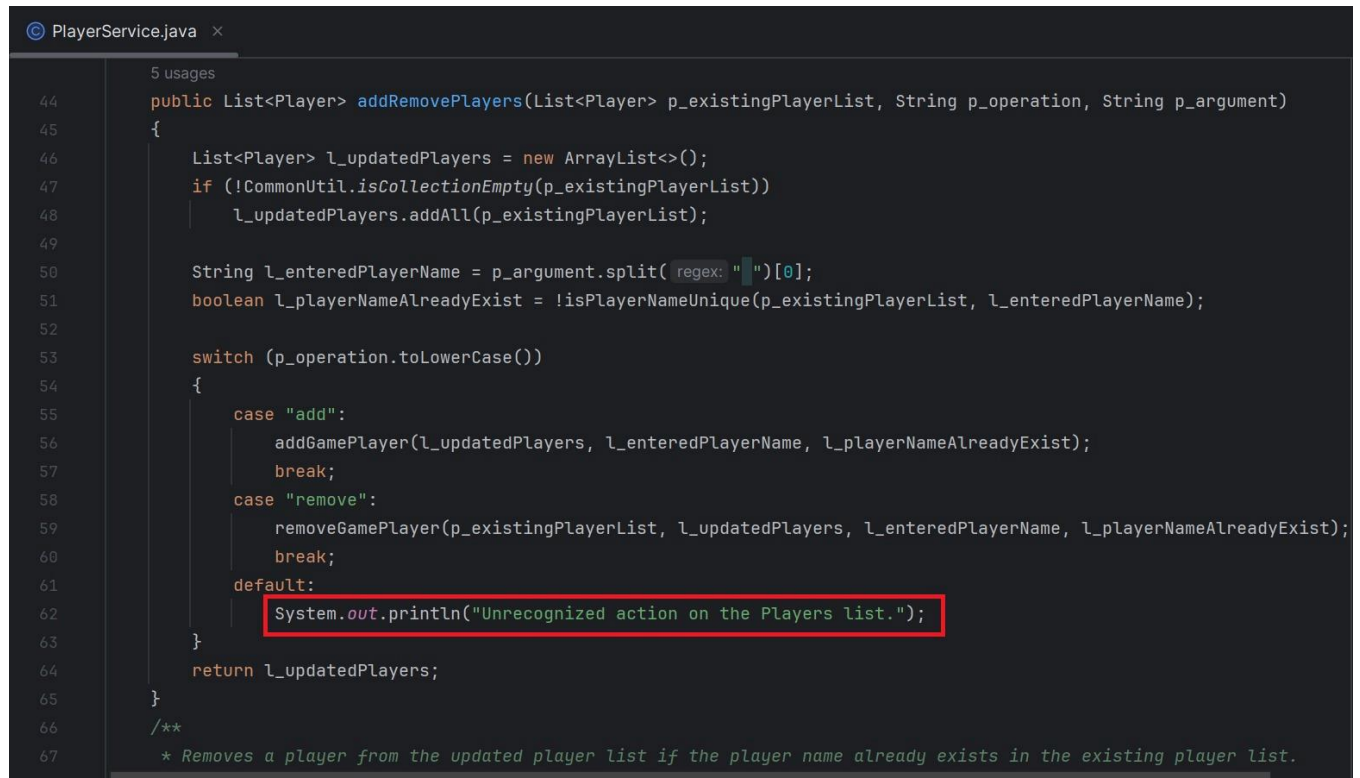
After: With the State pattern, the GameEngine class should now have a Phase field representing the current phase of the game. Each phase is encapsulated within its own class (StartupPhase, IssueOrderPhase, OrderExecutionPhase, etc.), implementing a common Phase interface or abstract class. The GameEngine class delegates phase-specific behavior to the current phase object.

```
GameEngine.java x
26  *
27  * @param p_phase new Phase to set in Game context
28  */
29  2 usages
30  private void setD_CurrentPhase(Phase p_phase){
31      d_currentPhase = p_phase;
32  }
33
34  /**
35  * this methods updates the current phase to Issue Order Phase as per State Pattern.
36  */
37  2 usages
38  public void setIssueOrderPhase(){
39      this.setD_gameEngineLog( p_gameEngineLog: "Issue Order Phase", p_logType: "phase");
40      setD_CurrentPhase(new IssueOrderPhase( p_gameEngine: this, d_gameState));
41      getD_CurrentPhase().initPhase();
42  }
43
44  /**
45  * this methods updates the current phase to Order Execution Phase as per State Pattern.
46  */
47  1 usage
48  public void setOrderExecutionPhase(){
49      this.setD_gameEngineLog( p_gameEngineLog: "Order Execution Phase", p_logType: "phase");
50      setD_CurrentPhase(new OrderExecutionPhase( p_gameEngine: this, d_gameState));
51      getD_CurrentPhase().initPhase();
52  }
```

2. Implementation of a game log file using the Observer pattern:

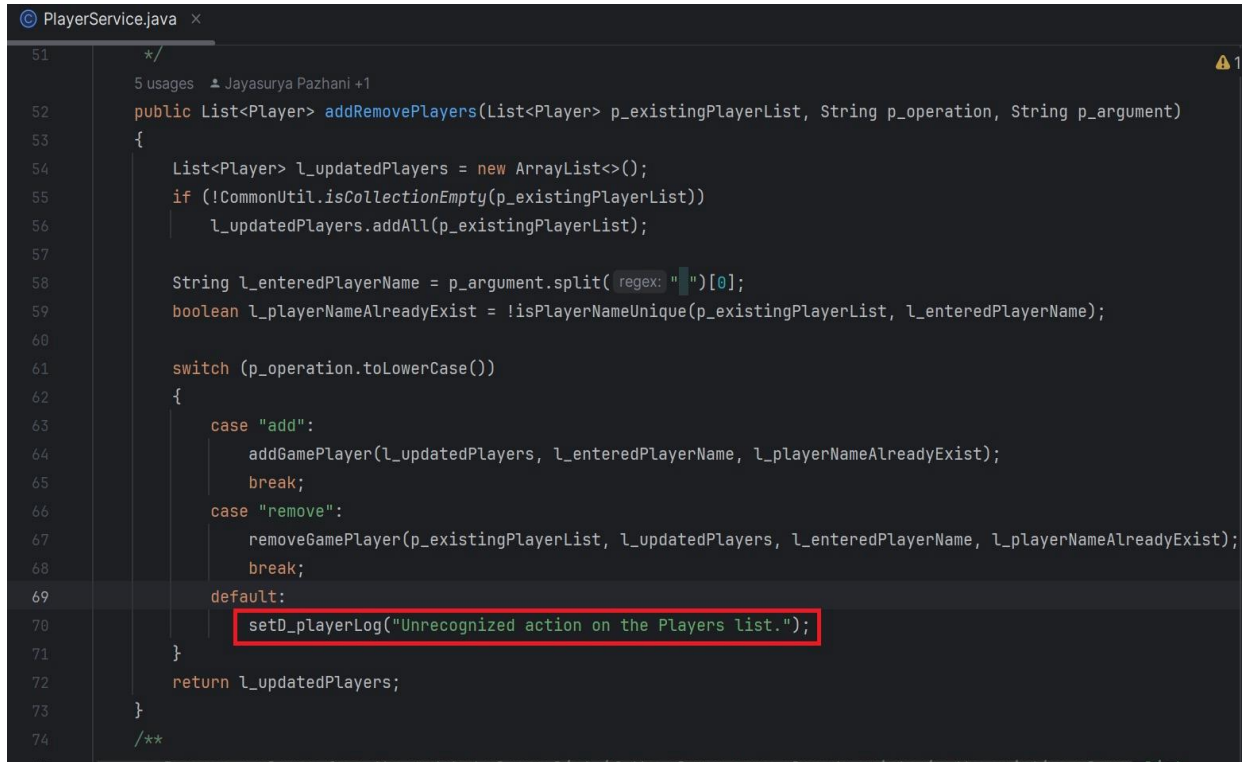
Before: In the original version, when an unrecognized action is encountered, a message is printed to the console using `System.out.println`.

Before



```
5 usages
44 public List<Player> addRemovePlayers(List<Player> p_existingPlayerList, String p_operation, String p_argument)
45 {
46     List<Player> l_updatedPlayers = new ArrayList<>();
47     if (!CommonUtil.isEmpty(p_existingPlayerList))
48         l_updatedPlayers.addAll(p_existingPlayerList);
49
50     String l_enteredPlayerName = p_argument.split(" ")[0];
51     boolean l_playerNameAlreadyExist = !isPlayerNameUnique(p_existingPlayerList, l_enteredPlayerName);
52
53     switch (p_operation.toLowerCase())
54     {
55         case "add":
56             addGamePlayer(l_updatedPlayers, l_enteredPlayerName, l_playerNameAlreadyExist);
57             break;
58         case "remove":
59             removeGamePlayer(p_existingPlayerList, l_updatedPlayers, l_enteredPlayerName, l_playerNameAlreadyExist);
60             break;
61         default:
62             System.out.println("Unrecognized action on the Players list.");
63     }
64     return l_updatedPlayers;
65 }
66 /**
67  * Removes a player from the updated player list if the player name already exists in the existing player list.
```

After: In the refactored version, instead of directly printing the message to the console, a method `setD_playerLog` is called to set a log message. This change likely indicates a broader architectural decision to manage and handle log messages more systematically within the codebase.



```
51  */
52  5 usages  Jayasurya Pazhani +1
53  public List<Player> addRemovePlayers(List<Player> p_existingPlayerList, String p_operation, String p_argument)
54  {
55      List<Player> l_updatedPlayers = new ArrayList<>();
56      if (!CommonUtil.isEmpty(p_existingPlayerList))
57          l_updatedPlayers.addAll(p_existingPlayerList);
58
59      String l_enteredPlayerName = p_argument.split(regex: " ")[0];
60      boolean l_playerNameAlreadyExist = !isPlayerNameUnique(p_existingPlayerList, l_enteredPlayerName);
61
62      switch (p_operation.toLowerCase())
63      {
64          case "add":
65              addGamePlayer(l_updatedPlayers, l_enteredPlayerName, l_playerNameAlreadyExist);
66              break;
67          case "remove":
68              removeGamePlayer(p_existingPlayerList, l_updatedPlayers, l_enteredPlayerName, l_playerNameAlreadyExist);
69              break;
70          default:
71              setD_playerLog("Unrecognized action on the Players list.");
72      }
73      return l_updatedPlayers;
74  }
75  /**
```

After

3. Use Command pattern to implement the Orders:

Before: In the "before" version, the code directly handles orders within the Player class. The `issueOrder` method takes an order type and details as parameters and executes the corresponding logic based on the order type. This approach can lead to tight coupling between order execution logic and the Player class, making it harder to extend or modify the code in the future.

Before

```
© Player.java x
189  /**
190   * Issues an order for the player
191   * @throws IOException If an I/O error occurs.
192   */
193   1 usage
194   public void issue_order() throws IOException {
195       BufferedReader l_reader = new BufferedReader(new InputStreamReader(System.in));
196       PlayerService l_playerService = new PlayerService();
197       System.out.println("\nPlease enter command to deploy reinforcement armies on the map for player : "
198           + this.getPlayerName());
199       String l_commandEntered = l_reader.readLine();
200       Command l_command = new Command(l_commandEntered);
201
202       if (l_command.getRootCommand().equalsIgnoreCase("deploy") && l_commandEntered.split(" ").length
203           == 2) {
204           l_playerService.createDeployOrder(l_commandEntered, l_player: this);
205       } else {
206           System.out.println(ApplicationConstants.INVALID_COMMAND_ERROR_DEPLOY_ORDER);
207       }
208   }
209
210  /**
211   * Returns the next order to be executed by the player.
212   * @return The next order, or null if no orders are pending.
213   */
214
215  2 usages
```

After: In the "after" version, the code utilizes the Command pattern to encapsulate orders as objects. The Player class's `issueOrder` method now takes an `Order` object as a parameter and adds it to the player's list of orders. This decouples the player logic from the specifics of order execution, promoting better separation of concerns.

After

A screenshot of an IDE window titled "Player.java" showing Java code. The code is as follows:

```
184     return p_player.get0_noOfUnallocatedArmies() < Integer.parseInt(p_noOfArmies) ? true : false;
185 }
186
187 @
188 public void issue_order(IssueOrderPhase p_issueOrderPhase) throws InvalidCommand, IOException, InvalidMap {
189     p_issueOrderPhase.askForOrder( p_player: this);
190 }
191
192 2 usages Jayanth Apagundi +1
193 public Order next_order() {
194     if (CommonUtil.isCollectionEmpty(this.order_list)) {
195         return null;
196     }
197     Order l_order = this.order_list.get(0);
198     this.order_list.remove(l_order);
199     return l_order;
200 }
```

The line `p_issueOrderPhase.askForOrder( p_player: this);` is highlighted with a red box. There is a lightbulb icon on line 193 and a loading spinner on line 195.

4. Identify and decouple dependencies:

Before: In the "checkPlayersAvailability" method, the before code displayed a message to the console when players were not available.

```
PlayerService.java x
102     p_updatedPlayers.add(l_addNewPlayer);
103     System.out.println("Player with name : " + p_enteredPlayerName + " has been added successfully.");
104 }
105 }
106
107 /**
108  * Check whether players are loaded or not.
109  *
110  * @param p_gameState current game state with map and player information
111  * @return boolean players exists or not
112  */
113 @ 3 usages
114 public boolean checkPlayersAvailability(GameState p_gameState) {
115     if (p_gameState.getD_players() == null || p_gameState.getD_players().isEmpty()) {
116         System.out.println("Kindly add players before assigning countries");
117         return false;
118     }
119     return true;
120 }
```

After: In the after code, this console output was removed to decouple the method from the specific output mechanism.

```
GameState.java  PlayerService.java x
148 }
149 /**
150  * Assigns countries to players in the game state. Each player is assigned a set number of countries
151  * based on the total number of countries available and the number of players.
152  * Additionally, assigns continents to players based on the countries they own.
153  *
154  * @param p_gameState The game state containing the map and the list of players.
155  */
156 2 usages Shuvanidhi +1
157 public void assignCountries(GameState p_gameState)
158 {
159     if (!checkPlayersAvailability(p_gameState)) {
160         p_gameState.updateLog(p_logMessage: "Before assigning countries, players should be added.", p_logType: "effect");
161         return;
162     }
163 }
```


5. Enhancing Exception Handling:

Before: In the "before" version, the deleteCountry and deleteCountryNeighbours methods did not have any exception handling.

Before

```
Continent.java x
134      */
135      1 usage
136      public void deleteCountry(Country p_country){
137          if(d_countries==null){
138              System.out.println("No such Country Exists");
139          }else {
140              d_countries.remove(p_country);
141          }
142      }
143
144      /**
145       * Deletes the neighboring relationship of a country with the specified ID within the continent.
146       * @param p_countryId The ID of the country whose neighbors are to be deleted.
147       */
148      2 usages
149      public void deleteCountryNeighbours(Integer p_countryId){
150          if (null!=d_countries && !d_countries.isEmpty()) {
151              for (Country c: d_countries){
152                  if (!CommonUtil.isNull(c.d_adjacentCountryIds)) {
153                      if (c.getD_adjacentCountryIds().contains(p_countryId)){
154                          c.deleteNeighbour(p_countryId);
155                      }
156                  }
157              }
158          }
159      }
160  }
```

After: In the "after" version, both methods now declare that they throw an InvalidMap exception. This indicates that these methods may encounter issues related to map validation and provides a way to handle such exceptions elsewhere in the code.

After

```
Continent.java x
134      * @param p_country The country to delete.
135      */
136      1 usage  Shuvanidhi +1
137      public void deleteCountry(Country p_country) throws InvalidMap{
138          if(d_countries==null){
139              System.out.println("No such Country Exists");
140          }else {
141              d_countries.remove(p_country);
142          }
143      }
144
145      /**
146       * Deletes the neighboring relationship of a country with the specified ID within the continent.
147       * @param p_countryId The ID of the country whose neighbors are to be deleted.
148       */
149      2 usages  Shuvanidhi +1
150      public void deleteCountryNeighbours(Integer p_countryId) throws InvalidMap{
151          if (null!=d_countries && !d_countries.isEmpty()) {
152              for (Country c: d_countries){
153                  if (!CommonUtil.isNull(c.d_adjacentCountryIds)) {
154                      if (c.getD_adjacentCountryIds().contains(p_countryId)){
155                          c.deleteNeighbour(p_countryId);
156                      }
157                  }
158              }
159          }
160      }
161  }
```